

Una directory, e chi ha il permesso di leggerla

OpenLDAP popolato da vuoto, interrogato da un'altra macchina, e messo davanti alla stessa domanda due volte — una in anonimo e una autenticato — per mostrare che la risposta dipende da chi chiede. Da una coppia accesa.

Comando `qlab run ldap-lab`

VM `ldap-lab-server 192.168.100.1 · ldap-lab-client 192.168.100.2`

Directory `dc=ldap-lab,dc=local`

Esito `qlab test ldap-lab` → 6 esercizi, 28 controlli, tutti superati

1. Due macchine, e una directory vuota

```
--- server ---
ens3          UP          10.0.2.15/24 metric 100 fec0::5054:ff:fe12:3456/64
fe80::5054:ff:fe12:3456/64
ens4          UP          192.168.100.1/24 fe80::5054:ff:fe00:401/64
--- client ---
ens3          UP          10.0.2.15/24 metric 100 fec0::5054:ff:fe12:3456/64
fe80::5054:ff:fe12:3456/64
ens4          UP          192.168.100.2/24 fe80::5054:ff:fe00:402/64
```

active

```
$ what the directory holds before exercise 1
dn:
namingContexts: dc=ldap-lab,dc=local
```

slapd gira, ma `namingContexts` dice `dc=nodomain` — il default Debian per un pacchetto installato e mai configurato. C'è un server e dentro non c'è nessuna directory. L'esercizio 1 della guida rimedia con uno script:

```
$ bash ~/demo-setup.sh
Admin DN:      cn=admin,dc=ldap-lab,dc=local
Admin password: admin

Users: alice (alice123), bob (bob123), charlie (charlie123)
Group: developers (alice, bob, charlie)

phpLDAPadmin: run 'qlab ports' on the host to see the URL
(from host browser, login with admin DN above)

Try from the client VM:
ldapsearch -x -H ldap://192.168.100.1 -b "dc=ldap-lab,dc=local" "(uid=alice)"
```

2. Una directory è un albero, e il percorso è il nome

```
$ the whole directory, as a tree of distinguished names
dn: dc=ldap-lab,dc=local

dn: ou=users,dc=ldap-lab,dc=local
dn: ou=groups,dc=ldap-lab,dc=local

dn: uid=alice,ou=users,dc=ldap-lab,dc=local
dn: uid=bob,ou=users,dc=ldap-lab,dc=local
dn: uid=charlie,ou=users,dc=ldap-lab,dc=local
dn: cn=developers,ou=groups,dc=ldap-lab,dc=local
```

Il nome di ogni voce — il suo **distinguished name** — è il suo percorso completo nell'albero, letto dalla foglia in su. `uid=alice,ou=users,dc=ldap-lab,dc=local` è insieme l'identificatore e la

posizione; non esiste una chiave separata. Spostate una voce e il suo nome cambia: è per questo che rinominare in LDAP è un'operazione diversa dal modificare.

La radice qui è `dc=ldap-lab,dc=local`, costruita con i *domain component* — una vecchia convenzione che proietta un nome DNS su una radice d'albero. `ou` è un'unità organizzativa, che non è nient'altro che un contenitore che avete deciso di creare.

3. Una voce è un insieme di attributi, e sono le sue classi a deciderli

```
$ one entry in full
dn: uid=alice,ou=users,dc=ldap-lab,dc=local
objectClass: inetOrgPerson
objectClass: posixAccount
objectClass: shadowAccount
uid: alice
sn: Smith
givenName: Alice
cn: Alice Smith
displayName: Alice Smith
uidNumber: 10001
gidNumber: 10000
homeDirectory: /home/alice
loginShell: /bin/bash
mail: alice@ldap-lab.local
```

Vanno lette per prime le righe `objectClass`, perché sono lo schema: dichiarano cosa quella voce è, e fra tutte decidono quali attributi sono obbligatori e quali soltanto ammessi.

- `inetOrgPerson` porta i campi umani — `cn`, `sn`, `givenName`, `mail`.
- `posixAccount` porta quelli Unix — `uidNumber`, `gidNumber`, `homeDirectory`, `loginShell`.

Quella seconda classe è tutta la ragione per cui LDAP si usa per il login: sono esattamente i campi che contiene `/etc/passwd`. Una macchina configurata per consultare questa directory prende gli utenti da qui invece che da un file locale, e `alice` diventa lo stesso account ovunque.

Si noti anche che un attributo può comparire più volte — `objectClass` lo fa. Gli attributi LDAP sono per natura multivalore, a differenza di una colonna di database.

```
$ a group, and how membership is expressed
dn: cn=developers,ou=groups,dc=ldap-lab,dc=local
objectClass: posixGroup
cn: developers
gidNumber: 10000
memberUid: alice
memberUid: bob
memberUid: charlie
```

L'appartenenza è `memberUid`, elencata sul **gruppo**, non sugli utenti. Quindi «in quali gruppi sta alice» è una ricerca fra i gruppi, non un campo che si legge su alice. È l'opposto dell'intuizione che quasi tutti portano, ed è il motivo per cui esiste `memberOf` come overlay facoltativo che mantiene il puntatore inverso.

4. I filtri

```
$ (uid=*)                                every entry that has a uid
uid: alice
uid: bob
uid: charlie

$ (&(objectClass=posixAccount)(uidNumber>=10002))    AND, with a range
uid: alice
uidNumber: 10001
uid: bob
uidNumber: 10002
uid: charlie
uidNumber: 10003

$ (|(uid=alice)(uid=bob))                OR
uid: alice
uid: bob
```

I filtri LDAP sono in notazione prefissa: prima l'operatore, poi i suoi operandi, ciascuno fra parentesi. `(&(a)(b))` è AND, `(|(a)(b))` è OR, `(!(a))` è NOT. Si legge male all'inizio e si compone bene una volta accettata.

`(uid=*)` è un test di presenza — le voci che hanno quell'attributo, qualunque ne sia il valore. È anche il filtro utile più economico per scoprire che tipo di voci contiene una directory.

5. Dalla rete, e chi sta chiedendo

```
$ the same query, from the other machine over the network
dn: uid=alice,ou=users,dc=ldap-lab,dc=local
cn: Alice Smith
mail: alice@ldap-lab.local
```

La stessa interrogazione dall'altra macchina. Sul client non è installato niente oltre a `ldapsearch`: una directory è un servizio di rete, e questo è il modo normale di usarla.

```
$ anonymous bind (-x, no credentials)
dn: uid=alice,ou=users,dc=ldap-lab,dc=local

$ authenticated bind as the directory manager
dn: uid=alice,ou=users,dc=ldap-lab,dc=local
userPassword:: e1NTSEF9THA2NFRiZ204WDU3R1grZWsl a3B4dno30Tg3ZFQwQW0=
```

Ed ecco il punto. Le due interrogazioni chiedono `userPassword` sulla stessa voce:

- **in anonimo**, la voce torna con il dn e nient'altro — l'attributo è stato trattenuto, senza alcun errore;
- **autenticati come gestore della directory**, l'attributo c'è.

È il controllo d'accesso di LDAP che funziona come deve, e vale la pena notarne la forma: non vi viene detto che qualcosa è stato nascosto. Una ricerca anonima che restituisce meno attributi del previsto è il segnale normale che avreste dovuto autenticarvi.

Il valore è in base64 (`::` invece di `:` dopo il nome dell'attributo) e dentro c'è `{SSHA}` — un hash con sale, non la password.

```
directory reset for the exercises.  
dn:  
namingContexts: dc=ldap-lab,dc=local
```

6. Verifica

```
Exercise 1: All 7 checks passed  
Exercise 2: All 6 checks passed  
Exercise 3: All 3 checks passed  
Exercise 4: All 5 checks passed  
Exercise 5: All 3 checks passed  
Exercise 6: All 4 checks passed  
Exercises run:      6  
Exercises passed:   6  
Exercises failed:   0  
Exercises skipped: 0  
All exercises passed!
```

7. Cosa portarsi via

- Il distinguished name è il percorso. Identità e posizione sono la stessa cosa.
- `objectClass` è lo schema: decide quali attributi una voce può e deve avere.
- `posixAccount` è ciò che rende una directory utilizzabile per il login Unix.
- L'appartenenza sta sul gruppo come `memberUid`; per la vista inversa serve un overlay.
- I filtri sono in notazione prefissa. `(uid=*)` verifica la presenza.
- Un bind anonimo è un'identità vera con limiti veri, e vedersi negare un attributo ha lo stesso aspetto dell'attributo che non esiste.

`guide.md` del plugin porta gli esercizi: configurare il dominio a mano, costruire l'albero, aggiungere utenti e gruppi, interrogare dal client, phpLDAPAdmin e modificare le voci.